Assignment 3 - Data Cleaning and Preparation

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

sns.set_theme(style="whitegrid")
print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

In [3]:
```
# Task 1: Import dataset
df = pd.read_csv('Telecom-Customer-Churn - Telecom-Customer-Churn.csv')

# Task 2: Explore dataset structure and summary
print(f"Dataset Shape: {df.shape}")
print("\n--- Column Info & Data Types ---")
df.info()

print("\n--- First 5 Rows ---")
display(df.head())

print("\n--- Summary Statistics (Numeric) ---")
display(df.describe())
```

```
Dataset Shape: (7043, 21)

--- Column Info & Data Types ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7032 non-null   float64
 20  Churn             7043 non-null   object 
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB

--- First 5 Rows ---
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSe
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

```
--- Summary Statistics (Numeric) ---
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7032.000000
mean	0.162147	32.371149	64.761692	2283.300441
std	0.368612	24.559481	30.090047	2266.771362
min	0.000000	0.000000	18.250000	18.800000
25%	0.000000	9.000000	35.500000	401.450000
50%	0.000000	29.000000	70.350000	1397.475000
75%	0.000000	55.000000	89.850000	3794.737500
max	1.000000	72.000000	118.750000	8684.800000

In [4]:
```
# Task 5: Standardize column names (lowercase, no leading/trailing spaces)
```

```
df.columns = df.columns.str.strip()

# Drop unique identifier if present (customerID doesn't help in churn prediction)
if 'customerID' in df.columns:
    df.drop(columns=['customerID'], inplace=True)

# Task 4: Remove duplicate records
duplicates_count = df.duplicated().sum()
df = df.drop_duplicates()
print(f"Removed {duplicates_count} duplicate rows.")

# Task 6: Convert columns to correct data types
# Fix 'TotalCharges' if stored as text/object due to whitespace strings
if 'TotalCharges' in df.columns and df['TotalCharges'].dtype == 'object':
    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'].str.strip(), errors='coerce')

# Task 5: Standardize string column values (strip extra whitespace)
str_cols = df.select_dtypes(include='object').columns
for col in str_cols:
    df[col] = df[col].astype(str).str.strip()

print("Data types successfully aligned!")
```

```
Removed 22 duplicate rows.
Data types successfully aligned!
```

In [5]:
```
# Check for nulls (including newly created NaNs from numeric conversion)
print("Missing values per column before handling:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# Impute numeric columns with median
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Impute categorical columns with mode
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
```

```
    df[col] = df[col].fillna(df[col].mode()[0])

print(f"\nRemaining missing values: {df.isnull().sum().sum()}")
```

```
Missing values per column before handling:
TotalCharges    11
dtype: int64

Remaining missing values: 0
```

In [6]:
```
# Identify outliers using IQR on key numeric continuous features
numeric_features = [col for col in ['tenure', 'MonthlyCharges', 'TotalCharges'] if col in df.columns]

for col in numeric_features:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap (winsorize) values beyond the boundaries
    df[col] = np.clip(df[col], lower_bound, upper_bound)

print("Outlier capping (IQR method) completed for numeric features.")
```

```
Outlier capping (IQR method) completed for numeric features.
```

In [7]:
```
# 1. Create Tenure Cohort / Group
if 'tenure' in df.columns:
    df['tenure_group'] = pd.cut(
        df['tenure'],
        bins=[-1, 12, 24, 48, 72],
        labels=['0-1 yr', '1-2 yrs', '2-4 yrs', '4+ yrs']
    )

# 2. Derive average monthly spend ratio (if both charges exist)
if 'TotalCharges' in df.columns and 'tenure' in df.columns:
    df['avg_monthly_spend'] = df['TotalCharges'] / (df['tenure'] + 1)
```

```
# 3. Standardize target column 'Churn' to binary numeric (0 and 1)
if 'Churn' in df.columns:
    df['Churn'] = df['Churn'].apply(lambda x: 1 if str(x).lower() == 'yes' else 0)

print("Engineered new features: 'tenure_group', 'avg_monthly_spend', and encoded 'Churn'.")
df.head(3)
```

```
Engineered new features: 'tenure_group', 'avg_monthly_spend', and encoded 'Churn'.
```

Out[7]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	Onl
0	Female	0	Yes	No	1	No	No phone service	DSL	No	
1	Male	0	No	No	34	Yes	No	DSL	Yes	
2	Male	0	No	No	2	Yes	No	DSL	Yes	

3 rows × 22 columns

In [8]:
```
# Separate features (X) and target variable (y)
X = df.drop(columns=['Churn']) if 'Churn' in df.columns else df.copy()
y = df['Churn'] if 'Churn' in df.columns else None

# One-hot encode remaining categorical variables
X = pd.get_dummies(X, drop_first=True)

# Normalize/scale numeric features using StandardScaler
scaler = StandardScaler()
scale_cols = [c for c in ['tenure', 'MonthlyCharges', 'TotalCharges', 'avg_monthly_spend'] if c in X.columr
X[scale_cols] = scaler.fit_transform(X[scale_cols])

print(f"Processed feature matrix shape: {X.shape}")
```

```
Processed feature matrix shape: (7021, 34)
```

In [9]:
```
if y is not None:
```

```
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    print(f"X_train shape: {X_train.shape}")
    print(f"X_test shape:  {X_test.shape}")
    print(f"y_train shape: {y_train.shape}")
    print(f"y_test shape:  {y_test.shape}")
```

```
X_train shape: (5616, 34)
X_test shape:  (1405, 34)
y_train shape: (5616,)
y_test shape:  (1405,)
```

In [10]:

```
# Combine X and y to save a fully prepared analytical dataset
cleaned_df = X.copy()
if y is not None:
    cleaned_df['Churn'] = y

cleaned_df.to_csv('cleaned_telecom_churn.csv', index=False)
print("Saved final cleaned and encoded dataset to 'cleaned_telecom_churn.csv'!")
```

```
Saved final cleaned and encoded dataset to 'cleaned_telecom_churn.csv'!
```